05 Окружение и Docker

Из чего состоит локальный запуск

Python работает на компьютере. PostgreSQL — в контейнере Docker.



↳ Два процесса общаются по сети. У каждого свои зависимости, настройки и журнал ошибок.

venv изолирует библиотеки проекта

Одна папка проекта — своё окружение и свой набор зависимостей.

Терминал · macOS / Linux — Учебный пример

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip --version
python -c "import sys;
print(sys.executable)"
```

До активации

`python` выбирается из системного PATH.

После активации

`python` указывает на `.venv/bin/python`.

↳ python -m pip устанавливает пакеты для того Python, которым вы запускаете приложение.

requirements фиксирует зависимости

Сохраняем проверенные версии, чтобы повторить окружение.

Фрагмент проверенного requirements.txt — Учебный пример

```
fastapi==0.141.1
pydantic==2.13.5
uvicorn==0.52.4
httpx==0.28.1
```

Установка из файла — Учебный пример

```
python -m pip install -r
requirements.txt
python -m pip check
```

Для полного воспроизведения

Фиксируем также транзитивные пакеты и версию Python.

↳ Файл зависимостей — рецепт установки. Его совместимость подтверждается запуском и проверками.

Образ — шаблон для контейнера

postgres:17 содержит подготовленное окружение PostgreSQL.

ОБРАЗ POSTGRES:17

- PostgreSQL
- Системные библиотеки
- Инструкция запуска

Скачать образ — Учебный пример

```
docker pull postgres:17
```

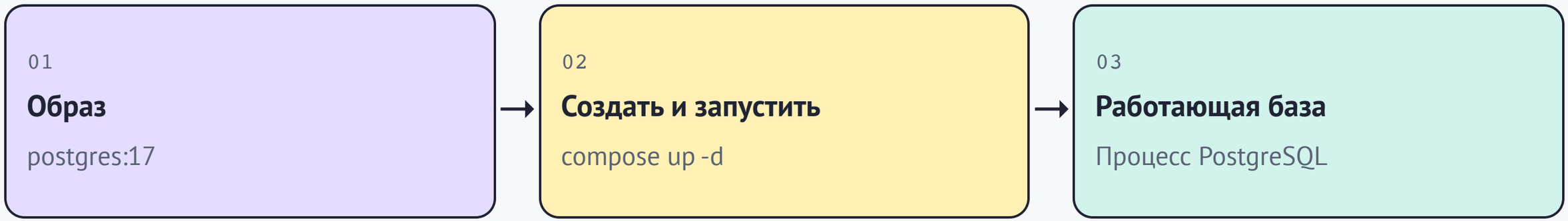
Образ не равен данным магазина

Из одного образа можно запустить несколько отдельных контейнеров.

↳ Образ содержит файлы и настройки запуска. Работающий процесс появится в контейнере.

Контейнер — запущенный экземпляр

У контейнера есть состояние, сеть и журнал процесса.



Команды из папки с compose.yaml — Учебный пример

```
docker compose ps
docker compose logs --tail 30 db
docker compose stop db
```

↳ stop останавливает процесс. Следующий up -d снова поднимает сервис.

Compose описывает запуск в файле

Один сервис db: образ, переменные, порт и место хранения.

compose.yaml · фрагмент — Учебный пример

```
services:
  db:
    image: postgres:17
    environment:
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    ports:
      - "127.0.0.1:55432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
```

Отступы задают структуру

db — имя сервиса.
pgdata — именованный том.

Запуск полного файла из примеров — Учебный пример

```
docker compose config --quiet
docker compose up -d --wait
```

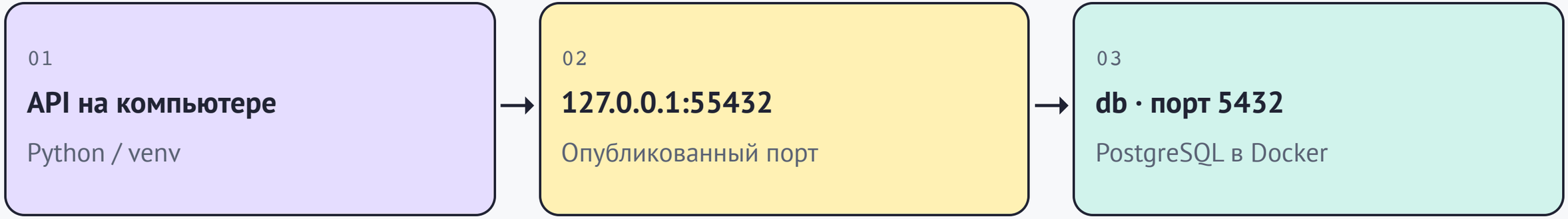
↳ Полный файл рядом с презентацией также объявляет pgdata, пользователя, базу и healthcheck.

05 Окружение и Docker

Адрес зависит от места запуска API

localhost всегда означает текущее сетевое окружение.

API запущен: На компьютере — наш стенд



Публикация порта: `127.0.0.1:55432:5432` · компьютер → контейнер

↳ С компьютера: 127.0.0.1:55432. Между сервисами одной сети Compose: db:5432.

Том переживает замену контейнера

Файлы PostgreSQL находятся в именованном томе pgdata.

Контейнер A

PostgreSQL пишет файлы базы

Контейнер B

После пересоздания читает те же файлы

↔

Том pgdata

Каталог данных PostgreSQL
таблицы · записи · индексы

В PostgreSQL 17 монтируем том в `/var/lib/postgresql/data`.

↳ Пересоздание контейнера и удаление тома — разные операции. Том не заменяет резервную копию.

DATABASE_URL — адрес подключения

Одна строка отвечает на вопросы: чем, кем, куда и в какую базу.

Учебный адрес · пароль вымышленный — Учебный пример

```
postgresql://lesson:lesson-only@127.0.0.1:55432/sneaker_lesson
```

`postgresql`
Протокол

`lesson`
Пользователь

`lesson-only`
Пароль

`127.0.0.1:55432`
Хост и порт

`sneaker_lesson`
Имя базы

↳ Один хост может обслуживать несколько баз. Имя базы — не имя таблицы и не имя контейнера.

.env хранит настройки окружения

Пример файла объясняет формат. Реальный файл подставляет значения.

.env.example · учебные значения — Учебный пример

```
POSTGRES_USER=lesson
POSTGRES_PASSWORD=lesson-only
POSTGRES_DB=sneaker_lesson
DB_PORT=55432
```

Сначала скопировать — Учебный пример

```
cp .env.example .env
```

Compose

Читает .env для подстановки `${...}` в YAML.

Python

Нужна явная загрузка настроек: например, `load_dotenv()` перед `os.environ["DATABASE_URL"]`.

↳ .env исключаем из Git. .env.example коммитим без реальных секретов.

API запущен — а база доступна?

Проверка готовности должна действительно обратиться к PostgreSQL.

Псевдокод GET /health — Учебный пример

```
попробовать:
    открыть соединение с таймаутом
    выполнить SQL: SELECT 1
    закрыть соединение
    вернуть 200, {status: "ok"}
если база недоступна:
    вернуть 503, {status: "unavailable"}
```

GET /live → 200

Процесс API отвечает. Базу не проверяем.

GET /health → 200

Наш readiness-маршрут проверил соединение и простой SQL-запрос.

↳ SELECT 1 проверяет соединение и выполнение SQL. Наличие таблиц и миграций проверяем отдельно.

Если не запускается: идём по цепочке

Проверяем ближайшую причину, затем повторяем тот же запрос.

1. Docker недоступен — `docker info`

Запустить Docker Desktop / engine

2. Контейнер не healthy — `docker compose ps`

Прочитать logs db; проверить порт

3. Нет соединения — Хост → порт → база

Сверить адрес с местом запуска API

4. Отказ в доступе — Роль и пароль

Проверить учётные данные уже созданной БД

5. Модуль не найден — `python -m pip --version`

Проверить venv и requirements

↳ Ошибка — подсказка, на каком участке оборвалась цепочка. Исправляем одну причину за раз.

Почему списка Python недостаточно

Каталог должен пережить перезапуск и быть общим для всех покупателей.

Память одного процесса — Учебный пример

```
products = [
    {"id": 10, "stock": 3}
]
# После перезапуска создаётся заново
```

Несколько процессов

У каждого — своя память и своя копия списка.

PostgreSQL

Хранит данные на диске и обслуживает разные соединения.

Правила согласованности

Ограничения и транзакции помогают сохранять корректные данные.

↳ База хранит общее состояние. Без корректных запросов и транзакций она не решает гонки автоматически.

Таблица: строки и столбцы

Строка описывает один товар. Столбец хранит один вид значения.

PRODUCTS

id	name	brand_id	price
10	North Runner	1	7990
20	City Walk	2	5990
30	North Trail	1	9990

3 строки → 3 товара

price → цена каждой строки

✓ Выполнено в PostgreSQL

↳ Таблица задаёт общую структуру. Каждая запись получает свои значения полей.

Тип поля объясняет его значение

Цена, количество, текст и время требуют разных правил хранения.

`text`

name

North Runner

Текст

`numeric(10,2)`

price

7990.00

Точное десятичное число

`integer`

stock

3

Целое количество

`boolean`

is_admin

false

Да / нет

`timestamptz`

created_at

2026-10-01 12:00+00

Момент времени

`NULL`

image_url

нет значения

Не пустая строка и не 0

↳ Тип ограничивает форму значения. CHECK и NOT NULL добавляют правила к этому типу.

Первичный ключ отличает записи

Название может повториться. ID должен однозначно найти нужную строку.

ОТДЕЛЬНЫЙ МИНИ-ПРИМЕР · ОДИНАКОВЫЕ НАЗВАНИЯ

id	name
10	Runner
20	Runner

PRIMARY KEY · первичный ключ — Учебный пример

```
id integer GENERATED BY DEFAULT
AS IDENTITY PRIMARY KEY
```

Найти одну конкретную запись

```
WHERE id = 20
```

Название для этого ненадёжно.

↳ PRIMARY KEY означает уникальность и запрет NULL. ID не обязан идти без пропусков.

Семь таблиц — семь задач магазина

Разделяем сведения, которые меняются и используются по-разному.

`users`
Кто покупает

`brands`
Кто выпускает

`products`
Что продаём

`product_sizes`
Размер и остаток

`cart_items`
Что выбрано сейчас

`orders`
Факт покупки

`order_items`
Состав и цена покупки

↳ Связи соединяют таблицы обратно: от позиции корзины можно дойти до размера, товара и бренда.

Пользователи и бренды: первые записи

ID нужен для связей. Остальные поля описывают сущность.

USERS

id	email	is_admin
7	anya@example.com	false
8	boris@example.com	false

BRANDS

id	name
1	North
2	City

↳ UNIQUE(email) запрещает повтор значения email. UNIQUE(name) для бренда — решение нашего примера.

Товар и размер — разные сущности

Цена относится к карточке. Остаток относится к её размеру.

PRODUCTS

id	name	price
10	North Runner	7990

PRODUCT_SIZES

id	product_id	size	stock
101	**10**	42	3
102	**10**	43	0

↳ North Runner существует один раз. Его размеры 42 и 43 имеют независимые остатки: 3 и 0.

Внешний ключ связывает две таблицы

products.brand_id ссылается на существующий brands.id.

BRANDS

id	name
1	North
2	City

PRODUCTS

id	name	brand_id
10	North Runner	**1**
20	City Walk	**2**
30	North Trail	**1**

↳ brand_id = 1 → brands.id = 1 → North. Название бренда берём из связанной строки.

Один ко многим: где хранится ссылка

Внешний ключ находится на стороне «много».

brands · 1

North · id=1

1 → N

products · много

10 · North Runner

30 · North Trail

products · 1

North Runner · id=10

1 → N

product_sizes · много

101 · размер 42

102 · размер 43

↳ У многих товаров один brand_id. У многих размеров один product_id.

10, 42 и 101 — три разных значения

Чтобы положить размер в корзину, нужен ID записи размера.

01
product_id = 10
Товар North Runner

→

02
size = 42
Номер размера обуви

→

03
id = 101
Запись product_sizes

Тело запроса в корзину — Учебный пример

```
{"product_size_id": 101, "quantity": 2}
```

↳ В API передаём product_size_id=101. Число 42 само по себе не определяет товар.

Корзина хранит выбор пользователя

Одна строка: кто выбрал, какой размер и сколько пар.

CART_ITEMS

id	user_id	product_size_id	quantity
55	**7**	**101**	2
56	**7**	**201**	1
57	**8**	**102**	1

Пользователь 7 → позиции 55 и 56

Пользователь 8 → позиция 57

↳ Связь user_id хранит владельца. Проверку доступа к его строкам должен выполнять сервер.

Заказ сохраняет факт покупки

Шапка описывает покупку целиком. Позиции фиксируют её состав и цены.

orders · id=501

user_id: **7** · completed
total: **21 970 ₽**

Сумма при оформлении

2 × 7 990 + 1 × 5 990
= **21 970 ₽**

ORDER_ITEMS · ORDER_ID=501

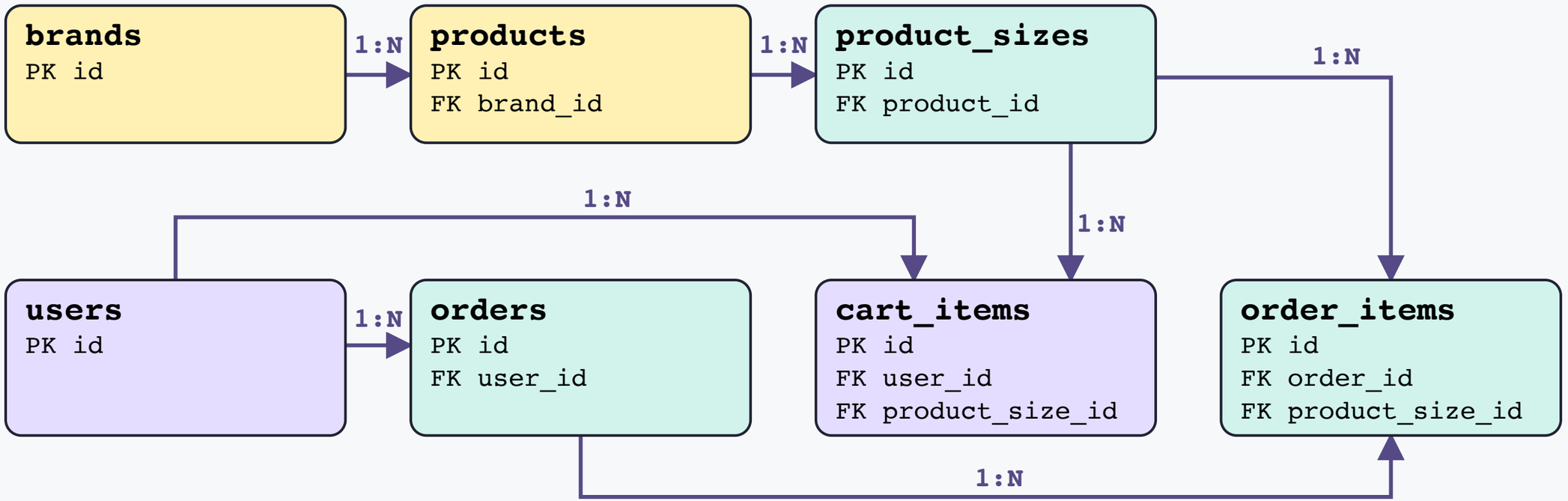
id	product_size_id	quantity	price_at_purchase
601	**101**	2	7990
602	**201**	1	5990

↳ price_at_purchase — цена одной пары на момент покупки. История не пересчитывается по текущему каталогу.

Вся схема магазина на одном экране

PK — первичный ключ. FK — внешний ключ. Стрелка: родитель → дочерние строки.

Подсветить связи: Вся схема 1:N — один ко многим



↳ Каждая стрелка — связь по ID. У родителя может быть 0…N дочерних строк.

Ограничения защищают саму базу

Проверки работают и при записи через API, и при прямом SQL.

NOT NULL `name = NULL`

Обязательное значение отсутствует

✓ Отклонено · 23502

UNIQUE `Второй бренд North`

Значение уже существует

✓ Отклонено · 23505

FOREIGN KEY `brand_id = 999`

Нет такого бренда

✓ Отклонено · 23503

CHECK `stock = -1`

Нарушено правило stock ⩾ 0

✓ Отклонено · 23514

Pydantic проверяет вход API. Ограничения PostgreSQL защищают данные независимо от способа записи.

06 PostgreSQL: таблицы и связи

SELECT читает нужные строки

Выбираем поля, фильтруем, сортируем и ограничиваем результат.

Выборка: Цена до 8 000

✓ Выполнено в PostgreSQL

SQL Учебный пример

```
SELECT id, name, price
FROM products
WHERE price <= 8000
ORDER BY price, id
LIMIT 10;
```

РЕЗУЛЬТАТ

id	name	price
20	City Walk	5990
10	North Runner	7990

↳ WHERE отбирает строки. ORDER BY задаёт порядок. LIMIT ограничивает количество строк.

JOIN соединяет строки по ключам

Добавим название бренда к каждому товару.

SQL · INNER JOIN — Учебный пример

```
SELECT p.id, p.name, b.name AS brand
FROM products AS p
JOIN brands AS b ON b.id = p.brand_id
ORDER BY p.id;
```

РЕЗУЛЬТАТ JOIN

id	name	brand
10	North Runner	**North**
20	City Walk	**City**
30	North Trail	**North**

↳ ON b.id = p.brand_id описывает совпадение ключей. p и b — короткие имена таблиц.

INSERT, UPDATE, DELETE меняют строки

Отдельный учебный бренд 99: добавили → переименовали → удалили.

1 · INSERT — Учебный пример

```
INSERT INTO brands(id, name)
VALUES (99, 'Demo');
```

ПОСЛЕ ШАГА

id	name
99	Demo

2 · UPDATE — Учебный пример

```
UPDATE brands
SET name = 'Demo Updated'
WHERE id = 99;
```

ПОСЛЕ ШАГА

id	name
99	Demo Updated

3 · DELETE — Учебный пример

```
DELETE FROM brands
WHERE id = 99;
```

ПОСЛЕ ШАГА

id	name

0 строк • [ ]

Показываем строку `WHERE id = 99` после выбранного шага.

↳ WHERE определяет, какие строки менять. Без него UPDATE и DELETE затронут все строки таблицы.

Практика: пройдите по связям

Позиция заказа 601. Найдите покупателя, товар, размер и бренд.

Кто: 601 → order 501 → user 7 → anya@example.com

Что: 601 → size 101 → product 10 → North Runner, размер 42

Бренд: product 10 → brand 1 → North

Цена покупки останется **7 990 ₽**. Текущая цена каталога её не заменяет.

↳ Готовы к следующему блоку: представить эти же таблицы и связи в Python через SQLAlchemy.